

Overview

After foundational MLOps concepts, the next challenge is operational execution: reproducible pipelines, predictable deployments, and actionable monitoring.

This chapter focuses on the practical architecture behind production ML services:

1. data and training pipelines,
2. deployment patterns (batch, online, streaming),
3. monitoring and alerting with drift-aware diagnostics.

ML Pipeline Architecture

Core pipeline layers

A production ML pipeline commonly contains:

- ingestion and validation,
- feature engineering,
- training and evaluation,
- artifact registration,
- deployment handoff.

Textual architecture diagram:

Data Sources -> Validation -> Feature Build -> Train -> Evaluate -> Register -> Deploy -> Monitor -> Retrain Trigger

Batch vs streaming pipelines

Batch

- periodic execution,
- simpler operations,
- suitable when real-time predictions are unnecessary.

Streaming

- near-real-time ingestion and updates,
- higher operational complexity,
- required for latency-sensitive or rapidly changing domains.

Hybrid

A common practical setup:

- batch retraining,
- online inference,
- delayed feedback loop for labels.

Deployment Patterns

Batch scoring

Used for:

- nightly risk scoring,
- weekly churn refresh,

- demand forecasting updates.

Trade-off:

- lower cost, higher prediction staleness.

Online serving (REST/gRPC)

Used for:

- user-facing personalization,
- fraud/risk checks at transaction time,
- low-latency recommendation APIs.

Trade-off:

- low latency demands stronger SRE discipline.

Canary and blue-green releases

Canary

- release model to a small traffic subset,
- compare performance and error metrics before broad rollout.

Blue-green

- maintain two full environments,
- switch traffic atomically,
- fast rollback by reverting traffic.

Monitoring in Production ML

Observability layers

System health

- p95/p99 latency,
- request throughput,
- error/timeout rates,
- resource utilization.

Data quality and drift

Monitor:

- missingness and schema deviations,
- feature distribution shift,
- population stability index (PSI) style shifts.

Model behavior

- prediction distribution drift,
- confidence score shift,
- delayed outcome-based performance tracking.

Business outcomes

- conversion lift,
- fraud loss rate,
- operational workload changes.

Drift taxonomy

- **Covariate drift**: feature distribution changes.
- **Label drift**: target prevalence changes.
- **Concept drift**: relationship between features and target changes.

Alert design

A useful alert has:

- clear threshold,
- severity level,
- owner and runbook,
- automatic context payload.

Avoid alert fatigue by combining persistence windows and multi-signal checks.

Pipeline and Deployment Testing Strategy

Test pyramid for ML ops

1. Unit tests for transformations.
2. Data contract tests for schema/freshness.
3. Pipeline integration tests.
4. Smoke inference tests post-deploy.
5. Canary acceptance checks.

Rollback readiness checklist

- previous stable model available,
- compatible feature schema,
- configuration switch path tested,
- incident protocol documented.

Cost and Performance Engineering

Inference efficiency

Optimize by:

- batching requests when possible,
- model quantization/pruning for suitable workloads,
- caching repeated predictions,
- feature store read optimization.

Pipeline efficiency

- incremental recomputation,
- partition-aware data processing,
- artifact reuse when upstream unchanged.

Production Case Studies & Exceptions

Case 1: Real-time fraud detection API

- initial deployment caused high false positives in one geography.
- canary comparison and feature-slice analysis identified regional feature drift.

Response:

- staged rollout with segment-specific thresholds.

Case 2: Batch churn model with weak refresh discipline

- monthly retraining lagged behind pricing/product changes.
- retention campaigns targeted stale risk scores.

Response:

- weekly retraining and freshness alerts.

Case 3 (Exception): High-throughput low-value endpoint

- strict p99 latency requirement but low business sensitivity to small quality changes.

Decision:

- use lighter model and periodic retraining; prioritize reliability and cost.

Interview Questions & Answers

1. **What are the core components of an ML pipeline?**
Data validation, feature engineering, training, evaluation, registration, deployment, and monitoring.
2. **When do you use batch vs online inference?**
Batch for non-urgent predictions; online for latency-critical decisions.
3. **What is canary deployment for models?**
A partial rollout to evaluate behavior before full traffic migration.
4. **What is blue-green deployment?**
Two parallel environments with traffic switching for fast rollback.
5. **How do you monitor data drift?**
Compare production and reference distributions using statistical distance metrics and thresholds.
6. **Difference between covariate and concept drift?**
Covariate drift is input shift; concept drift is change in mapping from inputs to target.
7. **What metrics should be monitored for online model serving?**
Latency, errors, throughput, drift indicators, prediction health, and business KPIs.
8. **How do you prevent data leakage in pipelines?**
Fit preprocessing on train split only and enforce temporal correctness.
9. **How do you design alerts for ML monitoring?**
Use severity, ownership, runbooks, and persistence windows to reduce noise.
10. **What is a feature contract?**
A guarantee of feature semantics, schema, and availability across training and serving.
11. **How do you choose rollback thresholds?**
Based on SLA violations and KPI degradation tolerance.

12. **Why do pipelines need integration tests?**
Because many failures occur at component boundaries, not inside individual steps.
13. **What is shadow deployment?**
Run a new model in parallel without affecting decisions, only for comparison.
14. **How do you reduce inference cost?**
Optimize model size, caching, batch processing, and architecture choices.
15. **What makes monitoring useful beyond dashboards?**
Actionable alerts tied to specific response playbooks.

References

- Google Cloud MLOps architecture and maturity guidance: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- AWS MLOps overview: <https://aws.amazon.com/what-is/mlops/>
- Orchestrate, Analyze, and Evaluate AI Deployments (Coursera): <https://www.coursera.org/learn/orchestrate-analyze-and-evaluate-ai-deployments>
- Evidently drift documentation: https://docs.evidentlyai.com/metrics/explainer_drift
- MLflow model registry concepts: <https://www.mlflow.org/docs/2.4.2/model-registry.html>
- DataTalksClub MLOps Zoomcamp (deployment and monitoring practice patterns): <https://github.com/DataTalksClub/zoomcamp>